

**UNIVERSIDAD INDUSTRIAL DE SANTANDER
FACULTAD DE INGENIERÍAS FISICOMECAÑICAS
ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA**

PLAN DE TRABAJO DE GRADO

FECHA DE PRESENTACIÓN: Bucaramanga, 8 de octubre de 2025

TÍTULO: Estudio experimental sobre el impacto del uso de SYCL en el cálculo de Streamlines, Streaklines y Pathlines

MODALIDAD: Trabajo de investigación

AUTORES: Juan Pablo Cerinza Zaraza, 2190081
Ashley Michelle Calderon Villamizar, 2162101

DIRECTOR: Sergio Augusto Gélvez Cortes, Escuela De Ingeniería De Sistemas e Informática

ENTIDAD INTERESADA: Universidad Industrial de Santander

TABLA DE CONTENIDO

1	INTRODUCCIÓN	2
2	PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA	2
3	OBJETIVOS	3
3.1	OBJETIVO GENERAL	3
3.2	OBJETIVOS ESPECÍFICOS	3
4	MARCO DE REFERENCIA	4
4.1	COMPUTACIÓN DE ALTO RENDIMIENTO (HPC)	4
4.1.1	COMPUTACIÓN HETEROGÉNEA	5
4.1.2	VISUALIZACIÓN DE CAMPOS DE FLUJO	6
4.1.3	ALGORITMOS DE INTEGRACION NUMERICA (RK4)	6
4.2	MODELOS DE PROGRAMACION EN PARALELO	7
4.3	SYCL (Y ESTANDARES DE PORTABILIDAD)	8
5	METODOLOGÍA	10
5.1	IMPLEMENTACIÓN DE RK4	10
5.2	DESARROLLO DEL AMBIENTE DE PRUEBAS	10
5.3	RECOPIACIÓN DE RESULTADOS	10
6	CRONOGRAMA	11
7	BIBLIOGRAFÍA	12

ESTUDIO EXPERIMENTAL SOBRE EL IMPACTO DEL USO DE SYCL EN EL CÁLCULO DE STREAMLINES, STREAKLINES Y PATHLINES

1 INTRODUCCIÓN

El desarrollo de la computación de alto rendimiento (High Performance Computing, HPC) ha permitido abordar problemas científicos y de ingeniería que requieren un procesamiento intensivo de datos. Dentro de este ámbito, la programación paralela constituye una estrategia esencial para aprovechar de manera eficiente las arquitecturas modernas de cómputo, particularmente aquellas basadas en unidades de procesamiento gráfico (GPU). Este enfoque resulta especialmente relevante en áreas como la dinámica de fluidos computacional (CFD, por sus siglas en inglés), en la cual el cálculo de trayectorias de partículas —representadas mediante Streamlines, Streaklines y Pathlines— demanda una elevada capacidad de cómputo y optimización en la gestión de recursos.

Entre las tecnologías predominantes en este campo, CUDA (Compute Unified Device Architecture) se ha consolidado como una herramienta de referencia debido a su estrecha integración con hardware de NVIDIA y a la madurez de su ecosistema de desarrollo. No obstante, la dependencia de un único proveedor limita su portabilidad y dificulta la adaptación a entornos heterogéneos. En respuesta a esta necesidad, ha emergido SYCL, un estándar desarrollado por el consorcio Khronos, que ofrece un modelo de programación en C++ para la ejecución paralela en dispositivos diversos (CPU, GPU, FPGA, entre otros). SYCL promueve la portabilidad y la interoperabilidad, características de creciente relevancia en la investigación y la industria, al tiempo que busca mantener un alto nivel de rendimiento.

El presente trabajo tiene como propósito realizar un estudio experimental sobre el impacto del uso de SYCL en el cálculo de Streamlines, Streaklines y Pathlines, comparando su rendimiento con una implementación equivalente desarrollada en CUDA. El análisis se centrará en métricas de tiempo de ejecución, utilización de memoria y eficiencia de cómputo, con el fin de establecer una comparación objetiva entre ambos enfoques. Para garantizar condiciones homogéneas, se emplearán herramientas de perfilamiento como NVIDIA Nsight Systems/Compute y entornos controlados mediante Docker, lo que permitirá asegurar la reproducibilidad de los experimentos.

Con ello, la investigación busca contribuir al conocimiento sobre la aplicabilidad de modelos de programación portables en escenarios de alto costo computacional, aportando evidencia que permita valorar la pertinencia de SYCL como alternativa a CUDA en el ámbito de la dinámica de fluidos y, en general, en el desarrollo de software científico de alto rendimiento.

2 PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA

La visualización de campos de flujo, mediante técnicas como Streamlines, Streaklines y Pathlines, es fundamental en diversas áreas de la ciencia e ingeniería, incluyendo la dinámica de fluidos computacional (CFD), la meteorología, y el modelado de procesos

físicos (Camp, Garth, Childs, Pugmire, y Joy, 2011). Estos métodos permiten representar de forma intuitiva el comportamiento de partículas en movimiento dentro de un campo vectorial, lo cual facilita el análisis y la interpretación de fenómenos complejos.

Sin embargo, la simulación precisa y eficiente de estas trayectorias implica un alto costo computacional, especialmente cuando se trabaja con volúmenes de datos grandes o se requieren resoluciones espaciales y temporales finas (Pugmire, Childs, Garth, Ahern, y Weber, 2009). En este contexto, la programación tradicional en C/C++, aunque eficiente, puede resultar insuficiente para explotar de manera óptima las capacidades de cómputo de los sistemas modernos, en especial aquellos que cuentan con aceleradores como GPUs.

En respuesta a esta necesidad, han surgido enfoques como la programación heterogénea, que busca distribuir la carga computacional entre distintos dispositivos de procesamiento, permitiendo una ejecución más rápida y eficiente. En particular, el estándar SYCL, basado en C/C++, ofrece una solución portable y flexible para desarrollar aplicaciones que aprovechen arquitecturas heterogéneas sin perder la expresividad y control del lenguaje de bajo nivel (Keryell, 2019).

A pesar de sus beneficios teóricos, existe una brecha de conocimiento práctico respecto al impacto real del uso de SYCL en problemas computacionalmente intensivos como el cálculo de líneas de flujo. No se cuenta con suficiente evidencia comparativa que cuantifique su desempeño frente a enfoques tradicionales en términos de tiempo de ejecución, consumo de memoria y escalabilidad, con respecto a esta técnica de visualización en particular. Para cuantificar el desempeño se realizan estudios experimentales usando métricas como tiempo de ejecución, uso de memoria y escalabilidad (Camp y cols., 2013), relacionadas con características del problema como: tamaño del dataset, tamaño del conjunto de semillas, distribución del conjunto de semillas y complejidad del campo vectorial (Camp y cols., 2011).

3 OBJETIVOS

3.1 OBJETIVO GENERAL

- Comparar el desempeño computacional de una técnica de visualización de campos de flujo (Streamlines, Streaklines y Pathlines), que involucra el método de Runge-Kutta implementado en lenguaje C/C++, mediante métodos tradicionales de cómputo y programación heterogénea con SYCL, evaluando métricas como el tiempo de ejecución, uso de memoria y escalabilidad.

3.2 OBJETIVOS ESPECÍFICOS

Aunque los OE incluidos no hacen parte de esta evaluación, se hace la observación que los mismos están formulados como actividades y no como Objetivos. NOTA: Se ha de tener presente esta observación en la formulación del Plan

- Formular e implementar una versión optimizada del algoritmo RK4 en SYCL, con el propósito de evaluar su desempeño en arquitecturas heterogéneas.
- Comparar el comportamiento de las implementaciones en C/C++ y en SYCL bajo diferentes configuraciones del problema (tamaño del dataset, conjunto de semillas y complejidad del campo vectorial), con el fin de identificar diferencias en su eficiencia y desempeño.
- Analizar comparativamente las ventajas, limitaciones y condiciones de uso de SYCL frente a implementaciones tradicionales en CPU y en GPU con CUDA, considerando métricas de tiempo de ejecución, uso de memoria y escalabilidad.

4 MARCO DE REFERENCIA

Como base para el desarrollo del proyecto es necesario establecer los fundamentos para la formulación del algoritmo optimizado de RK4, por lo cual es imperativo conocer las características del problema, los principios de la computación heterogénea y la computación de alto rendimiento, las aplicaciones de la misma en la industria y las partes requeridas para la optimización, las cuales se describen a continuación.

4.1 COMPUTACIÓN DE ALTO RENDIMIENTO (HPC)

La computación de alto rendimiento, conocida como HPC, forma parte indisoluble del ámbito de la computación y está dedicada fundamentalmente a la resolución de problemas, que requieren para su resolución un gran poder computacional. El objetivo de HPC es el de aprovechar arquitecturas avanzadas y paralelas que puedan ser utilizadas para resolver tareas que de ser necesarias en sistemas convencionales no se podrían realizar precisamente debido, entre otras consideraciones, a la complejidad de los cálculos a realizar y a la gran cantidad de datos a procesar.

De un modo general, HPC ha pasado a ser parte fundamental de la computación en diversas disciplinas científicas y de la ingeniería, permitiendo realizar simulaciones numéricas, el procesamiento de grandes cantidades de datos o el modelado detallado de fenómenos físicos. Este enfoque ha permitido la expansión de áreas como la predicción del tiempo (clima), la exploración espacial, la biología computacional, el diseño de nuevos materiales, etc.

A lo largo de la historia —de los sistemas HPC— las supercomputadoras se transforman en infraestructuras distribuidas y clústeres HPC y estos cambios han conducido también a la aparición de paradigmas de programación que permiten explotar el paralelismo como por ejemplo MPI (Message Passing Interface) y OpenMP (Open Multi-Processing) y más recientemente la aparición de las GPUs han permitido incrementar todavía más la capacidad computacional por medio de un paralelismo masivo orientado hacia cálculos científicos y simulaciones.

Un aspecto clave de los sistemas HPC en la actualidad es la migración hacia arquitecturas heterogéneas que integran diferentes tipos de procesadores (CPU, GPU, FPGA y aceleradores específicos) con vista a una mayor eficiencia. El éxito que han ido logrando estas arquitecturas no responde solamente a la necesidad de un mayor poder computacional, sino que tiene que ver también en el avance del conocimiento de la factibilidad y del consumo energético, así como de la escalabilidad. En este marco, la programación y la portabilidad del código, se convierte en uno de los pilares que conduce a los avances que presentan nuevos modelos como SYCL, que intenta crear un acceso unificado a las diferentes plataformas, conservando al mismo tiempo el rendimiento (Reinders y cols., 2021) .

Por consiguiente, HPC no debe ser entendida solo como la existencia y uso de supercomputadoras, sino que forma parte de un conjunto de tipos de tecnología que siempre están evolucionando y que tienen mucha repercusión en la aceleración del conocimiento científico y la posibilidad de poder tratar la solución de problemas que tienen mucho significado a nivel mundial.

4.1.1 COMPUTACIÓN HETEROGÉNEA

La computación heterogénea tiene su fundamento en el uso conjunto de varios tipos de procesadores existentes en un mismo sistema y tiene como objetivo el aumento del rendimiento y de la eficiencia. En este modelo de computación no solo se hace funcionar la CPU, sino que se hacen funcionar otros dispositivos adicionales a esta como las GPU, FPGA u otros tipos de aceleradores, de forma que se distribuyen las tareas en función de las capacidades de cada una de las unidades de procesamiento.

El interés por este modelo ha crecido en los últimos años debido a que los problemas actuales no tienen únicamente que ver con la capacidad de cómputo, sino también con la eficiencia energética y la escalabilidad. Si bien las CPU ofrecen flexibilidad y son apropiadas para las tareas secuenciales, las GPU llegan a aportar niveles de paralelismo muy elevados, de tal forma que se pueden utilizar en simulaciones, cálculos numéricos masivos o grandes volúmenes de datos. (Mittal y Vetter, 2015).

La adopción de arquitecturas heterogéneas se ha visto favorecida por la aparición de entornos de programación como CUDA, OpenCL y SYCL, que facilitan su programación y proporcionan diferentes niveles de portabilidad. (Keryell, 2019). Aun así, la computación heterogénea tiene también sus inconvenientes, como la necesidad de encontrar algoritmos a partir de los cuales las computaciones puedan adaptarse a hardware heterogéneo y la gestión de la carga de trabajo en los distintos dispositivos.

Este paradigma se ha instalado como una de las principales tendencias en la computación de alto rendimiento, debido a que permanece en la línea que se busca entre velocidad de ejecución, eficiencia energética y uso de recursos.

4.1.2 VISUALIZACIÓN DE CAMPOS DE FLUJO

La visualización de los campos de flujo es una metodología para realizar una representación gráfica del comportamiento de los sistemas dinámicos que se desarrollan con respecto al tiempo y al espacio. Un campo de flujo es la descripción de cómo varía una magnitud vectorial, como puede ser, por ejemplo, la velocidad o la fuerza, en los distintos puntos de un dominio, siendo su análisis importante en la dinámica de fluidos, la meteorología o bien en la física computacional.

La importancia de este concepto reside en que es capaz de entender fenómenos complejos mediante representaciones visuales (como diagramas de flujo) que ayudan en la interpretación de los datos numéricos. En problemas como la simulación de fluidos, la dispersión de contaminantes o el modelado atmosférico, las soluciones numéricas en sí mismas son difíciles de interpretar; por ello, se requiere de la visualización para que se puedan ver patrones, trayectorias y estructuras internas. (Laramee y cols., 2004).

Hay muchas aplicaciones de la visualización de campos de flujo. Por ejemplo, se emplea en el diseño aerodinámico de la industria del automóvil y la aeronáutica para pronosticar el clima; también se utiliza para simular la turbulencia o estudiar el movimiento de partículas en medios físicos. Igualmente, se utiliza para el análisis de modelos en biomedicina. A partir de todas estas aplicaciones, se requiere de métodos computacionales intensivos para poder resolver numéricamente el problema, como para convertir gráfica y visualmente los resultados. (Laramee y cols., 2004) (Post, Vrolijk, Hauser, Laramee, y Doleisch, 2003).

Sin embargo, la visualización de campos de flujo presenta limitaciones debido principalmente al alto coste computacional que se deriva de ciertos aspectos; por ejemplo, el cálculo de trayectorias, la interpolación de datos en malla tridimensional y el renderizado de grandes cantidades de datos requieren de sistemas que puedan ejecutar tareas en paralelo, y de arquitecturas de computación avanzada. (Post y cols., 2003).

Es en este sentido que desde el proyecto se ha considerado este tipo de problemas, ya que el algoritmo para la integración numérica RK4, optimizado y adaptado a arquitecturas heterogéneas, sirve para calcular trayectorias para partículas dentro de campos de flujo. Por lo que para la visualización no solo se quedaría como una aplicación representativa, sino que también serviría como un caso de estudio para poder validar la eficiencia y portabilidad de las implementaciones en SYCL frente a otras soluciones de forma tradicional.

4.1.3 ALGORITMOS DE INTEGRACION NUMERICA (RK4)

Los algoritmos de integración numérica constituyen un conjunto de métodos que se idearon para aproximar la solución de Ecuaciones Diferenciales Ordinarias (EDO), sobre todo, cuando no se puede obtener una solución analítica, exacta. Entre todos estos métodos, el método de Runge-Kutta de cuarto orden (RK4) es uno de los más utilizados a causa de su equilibrio entre sencillez, precisión y coste computacional. (Butcher, 2008).

La razón por la que este método es importante, radica en su capacidad para proporcionar resultados con un error relativamente pequeño sin tener que aumentar excesivamente el coste de cálculo. Por eso, el RK4 ha pasado a ser considerado un estándar en lo que se refiere a simulaciones físicas, ingeniería, biología computacional y en general cualquier disciplina que busque modelar fenómenos dinámicos mediante EDO. (Süli y Mayers, 2003).

Entre sus aplicaciones se encuentran la simulación de trayectorias de partículas en campos vectoriales, la dinámica de fluidos, la modelación de circuitos eléctricos, el análisis orbital en astrofísica y la biomecánica. En estas aplicaciones, el RK4 ofrece aproximaciones estables y confiables incluso en sistemas fuertemente no lineales.

Sin embargo, el método tiene una serie de limitaciones ligadas al coste en recursos computacionales, sobre todo cuando la dimensión de los datasets aumenta o cuando se busca una alta resolución tanto temporal como espacial. En estos casos, llevar a cabo el algoritmo de forma secuencial puede acabar convirtiéndose en un cuello de botella, lo que requiere recurrir a utilizar técnicas de paralelización y arquitecturas de computación más avanzadas. (Camp y cols., 2011)(Pugmire y cols., 2009).

En el caso del proyecto, RK4 se convierte en el núcleo matemático que se desea optimizar. Su implementación en un entorno heterogéneo gracias a SYCL permite trabajar en conseguir mejoras en cuanto a el rendimiento y la portabilidad al comparar sus resultados a implementaciones tradicionales en CPU y GPU, como las que basan en CUDA.

4.2 MODELOS DE PROGRAMACION EN PARALELO

La programación paralela es un paradigma que intenta ejecutar muchas operaciones a la vez, dividiendo un problema en subproblemas más pequeños que pueden resolverse de manera simultánea en diferentes unidades de procesamiento. Este modelo, a diferencia de la programación secuencial, trata de maximizar el uso de los recursos de hardware y es fundamental para hacer frente a nuevos problemas de gran escala en ciencia e ingeniería. (Grama et al. 2003).

Una vez más, su importancia reside en que esas capacidades del hardware moderno están conformadas por procesadores multinúcleo, tarjetas gráficas (GPU) y aceleradores. Es con la ejecución paralela que se puede mejorar los tiempos de cómputo y mejorar la eficiencia energética para las aplicaciones que requieren grandes volúmenes de cálculo, como es el caso de la simulación científica, el aprendizaje automático o la visualización de datos. (Kumar et al., 2022).

Existen diferentes modelos y herramientas de este paradigma. CUDA, desarrollado por NVIDIA, se ha convertido en un estándar de programación para GPU propietarias. OpenCL es un marco de programación abierto que permite usar arquitecturas heterogéneas. OpenMP permite la programación paralela en entornos de memoria compartida para CPU multinúcleo, y por último SYCL, promovido por Khronos Group, extiende el modelo de C++ para múltiples arquitecturas, pero en un marco portable y

estándar. (Khronos Group, 2020; Reinders et al., 2021).

No obstante, este método también conlleva ciertas limitaciones. Entre ellas están la dependencia del hardware —hay modelos que se ejecutan solamente en arquitecturas específicas—, la complejidad en el control del paralelismo y dependencias como la dificultad de portar aplicaciones entre plataformas heterogéneas. Este tipo de restricciones ha sido el impulso para el desarrollo de otros métodos que consideren una buena relación entre el rendimiento y la portabilidad. (Choi et al., 2020).

En la parte asociada a este proyecto, la paralelización de las aplicaciones se relaciona estrictamente con la elección de SYCL como modelo de referencia. Esta estrategia permite escribir código portable en C++ y ejecutarlo en CPU, GPU y FPGA, lo que resulta un aspecto fundamental a considerar para evaluar eficiencia y escalabilidad frente a otros modelos convencionales.

4.3 SYCL (Y ESTÁNDARES DE PORTABILIDAD)

SYCL (del inglés Standard C++ for Heterogeneous Computing) es un estándar abierto de Khronos Group destinado a la extensión del lenguaje C++ con el fin de facilitar la programación en arquitecturas heterogéneas. A diferencia de los modelos basados en implementaciones propietarias, SYCL desarrolla una interfaz de programación de alto nivel que permite a los desarrolladores escribir código portable de forma única para ejecutarse posteriormente en CPU, GPU, FPGA u otros aceleradores (Khronos Group, 2020).

La importancia del estándar SYCL se encuentra en su desarrollada portabilidad y además por su productividad. Por enlazarse de manera nativa con C++, permite a los programadores aplicar las características modernas de C++ sin tener que aprender un nuevo paradigma. Este mismo hecho ayuda a promover la interoperabilidad entre familias de plataformas que es especialmente importante en un contexto donde el hardware en la arquitectura puede variar entre diferentes generaciones y fabricantes (Reinders et al., 2021).

Entre sus aplicaciones más destacadas nos encontramos con proyectos de cómputo de alto rendimiento (HPC), simulaciones científicas, inteligencia artificial y para análisis de datos. Frameworks como oneAPI de Intel, DPC++, o hipSYCL, hacen de SYCL una buena base para ofrecer entornos de desarrollo con rendimiento y a la vez portabilidad, corroborando así su importancia en el marco de estándares modernos de programación paralela (Intel, 2022; Alpaya et al., 2020).

No obstante, SYCL también posee algunas limitaciones. La principal limitación radica en su madurez relativa comparada con los modelos ya consolidados, como CUDA, que deriva en un ecosistema de herramientas y bibliotecas todavía no desarrollado. El soporte de hardware tampoco es homogéneo, depende del compromiso que asuman cada uno de los fabricantes, lo que puede hacer que existan diferencias de rendimiento entre dispositivos (Choi et al., 2020).

Durante el transcurso de este proyecto, SYCL se establece como la herramienta relevante

para comprobar la portabilidad y el rendimiento de la implementación del algoritmo RK4 aplicado a la visualización de campos de flujo. Su uso permite no solo explorar las ventajas de trabajar con un estándar abierto también permite establecer comparativas respecto a otros modelos de tipo propietario en casos de heterogeneidad.

5 METODOLOGÍA

Para el desarrollo del trabajo de grado, se propone un plan de 3 fases, que acarrea una secuencia lógica desde la implementación hasta el análisis de los resultados:

- Implementación de RK4
- Desarrollo del ambiente de pruebas
- Recopilación de resultados

5.1 IMPLEMENTACIÓN DE RK4

Esta primera fase consta de la implementación del algoritmo RK4 para visualización de Streamlines, Streaklines y Pathlines en C/C++ usando las tecnologías a comparar, CUDA y SYCL. Que posteriormente serán sujetos de análisis y comparación en la fase 3.

ACTIVIDADES

- 5.1.1. Recopilación de datos de entrada de tamaño variable para pruebas.
- 5.1.2. Implementación del algoritmo RK4 en CUDA.
- 5.1.3. Implementación del algoritmo RK4 en SYCL.

5.2 DESARROLLO DEL AMBIENTE DE PRUEBAS

La segunda fase consta de la creación de un ambiente controlado donde se ejecutarán ambos algoritmos, manteniendo las mismas condiciones de cómputo para ambos algoritmos, y se recopilarán las métricas establecidas para la comparación.

ACTIVIDADES

- 5.2.1. Determinación de métricas a evaluar.
- 5.2.2. Implementación en Docker del ambiente de pruebas.
- 5.2.3. Ejecución síncrona de pruebas para ambas implementaciones.

5.3 RECOPIACIÓN DE RESULTADOS

Durante la tercera fase y última fase, se compilarán los resultados en un formato adecuado para su análisis, y se retroalimentarán los resultados obtenidos para concluir el trabajo de grado.

ACTIVIDADES

5.3.1. Organización de los resultados obtenidos en gráficos y tablas.

5.3.2. Retroalimentación y conclusiones de los resultados obtenidos.

6 CRONOGRAMA

En la tabla 1 se presenta el cronograma propuesto para el desarrollo del el proyecto. En este se establece un tiempo total de 16 semanas en las cuales se desarrollarán las actividades definidas en la metodología.

Fase / Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Implementación De RK4																
Desarrollo Del Ambiente De Pruebas																
Recopilación De Resultados																
Fase: Implementación De RK4																
Actividad / Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
5.1.1.																
5.1.2.																
5.1.3																
Fase: Desarrollo Del Ambiente De Pruebas																
Actividad / Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
5.2.1.																
5.2.2.																
5.2.3																
Fase: Recopilación De Resultados																
Actividad / Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
5.3.1.																
5.3.2.																

Tabla 1: Cronograma del proyecto

7 BIBLIOGRAFÍA

- Butcher, J. C. (2008). *Numerical methods for ordinary differential equations* (2. Aufl ed.). Chichester: Wiley.
- Camp, D., Garth, C., Childs, H., Pugmire, D., y Joy, K. (2011, noviembre). Streamline Integration Using MPI-Hybrid Parallelism on a Large Multicore Architecture. *IEEE Transactions on Visualization and Computer Graphics*, 17(11), 1702–1713. Descargado 2025-09-10, de <http://ieeexplore.ieee.org/document/5669297/> doi: 10.1109/TVCG.2010.259
- Camp, D., Krishnan, H., Pugmire, D., Garth, C., Johnson, I., Bethel, E. W., . . . Childs, H. (2013). *GPU Acceleration of Particle Advection Workloads in a Parallel, Distributed Memory Setting*. The Eurographics Association. Descargado 2025-09-10, de <http://diglib.eg.org/handle/10.2312/EGPGV.EGPGV13.001-008> doi: 10.2312/EGPGV/EGPGV13/001-008
- Keryell, R. (2019, noviembre). *SYCL A Single-Source C++ Standard for Heterogeneous Computing*. San José, California. Descargado de https://www.khronos.org/assets/uploads/developers/presentations/SC19-H2RC-keynote-SYCL_Nov19.pdf
- Laramée, R. S., Hauser, H., Doleisch, H., Vrolijk, B., Post, F. H., y Weiskopf, D. (2004, junio). The State of the Art in Flow Visualization: Dense and Texture-Based Techniques. *Computer Graphics Forum*, 23(2), 203–221. Descargado 2025-09-30, de <https://onlinelibrary.wiley.com/doi/10.1111/j.1467-8659.2004.00753.x> doi: 10.1111/j.1467-8659.2004.00753.x
- Mittal, S., y Vetter, J. S. (2015, enero). A Survey of Methods for Analyzing and Improving GPU Energy Efficiency. *ACM Computing Surveys*, 47(2), 1–23. Descargado 2025-09-30, de <https://dl.acm.org/doi/10.1145/2636342> doi: 10.1145/2636342
- Post, F. H., Vrolijk, B., Hauser, H., Laramée, R. S., y Doleisch, H. (2003, diciembre). The State of the Art in Flow Visualisation: Feature Extraction and Tracking. *Computer Graphics Forum*, 22(4), 775–792. Descargado 2025-09-30, de <https://onlinelibrary.wiley.com/doi/10.1111/j.1467-8659.2003.00723.x> doi: 10.1111/j.1467-8659.2003.00723.x
- Pugmire, D., Childs, H., Garth, C., Ahern, S., y Weber, G. H. (2009, noviembre). Scalable computation of streamlines on very large datasets. En *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (pp. 1–12). Portland Oregon: ACM. Descargado 2025-09-30, de <https://dl.acm.org/doi/10.1145/1654059.1654076> doi: 10.1145/1654059.1654076
- Reinders, J., Ashbaugh, B., Brodman, J., Kinsner, M., Pennycook, J., y Tian, X. (2021). *Data Parallel C++: Mastering DPC++ for Programming of Heterogeneous Systems using C++ and SYCL*. Berkeley, CA: Apress. Descargado 2025-10-07, de <http://link.springer.com/10.1007/978-1-4842-5574-2> doi: 10.1007/978-1-4842-5574-2
- Süli, E., y Mayers, D. F. (2003). *An Introduction to Numerical Analysis* (1.^a ed.). Cambridge University Press. Descargado 2025-09-30, de <https://www.cambridge.org/core/product/identifier/9780511801181/type/book> doi: 10.1017/CBO9780511801181